

Metro Ticket Generation System

Phase 4 — Data Migration, Testing & Security | ServiceNow

Phase Purpose	Validate the implemented solution, protect backend data and confirm reliable end-to-end behavior.
Primary Areas	Access Control, QA Testing, Data Integrity, validation, mandatory fields, auto-population and field mapping.
Security Approach	Role-based access and ServiceNow ACL evaluation while allowing clients to use the catalog without direct database access.
Expected Outcome	A tested, secure and consistent Metro Ticket solution ready for deployment and final documentation.

1. Phase 4 Overview

Phase 4 is the quality and security gate between development and deployment. The objective is not only to prove that the Metro Ticket form works, but also to verify that users receive the correct behavior while unauthorized users cannot directly manipulate protected backend records.

Configured Solution → Security Review → Functional QA → Validation → Data Integrity → End-to-End Verification → Deployment Ready

2. Phase 4 Brainstorming

- Can a normal client book a ticket without being granted direct database access?
- Can unauthorized roles read, write, create or delete backend records?
- Are Starting From and Going To always valid station values?
- Are fare values consistent with the selected route, journey type and passenger count?
- Can users submit incomplete or invalid requests?
- Do UI policies and client scripts behave correctly when values are changed multiple times?
- Does the QR ticket appear only through the intended checkout flow?
- Can the system preserve historical ticket information when master data changes?

3. Phase 4 Workstreams

Workstream	Objective
Access Control	Restrict database operations according to roles and application requirements.
QA Testing	Verify positive, negative, boundary and end-to-end scenarios.
Data Integrity	Ensure valid, complete and consistent station, fare, booking and ticket data.
Validation	Prevent invalid source/destination, passenger count, fare and payment inputs.
Field Mapping	Map catalog variables to backend fields and populate values correctly.

4. Access Control

The implementation uses role-based security to separate the catalog user experience from direct backend database access. Users/clients can book a ticket or recharge a Metro card through the intended interface, while database tables remain protected by ServiceNow access controls.

5. ACL Configuration

- When a custom table is created, ServiceNow can create default table-level ACL records for common operations.
- The observed configuration contains record-level controls for read, create, delete and write operations.
- These controls determine whether the current user/role can perform the requested database operation.
- In this project, the default ACLs created with the custom table are used rather than adding separate custom restriction ACLs.
- The catalog should remain the preferred route for clients to perform business actions instead of exposing raw database tables.

6. Observed ACL Structure

Operation	Type	Purpose
read	record	Controls whether a user can read records.
create	record	Controls creation of new records.
delete	record	Controls deletion of records.
write	record	Controls modification of existing records.

7. Security Design Principle

Catalog access ≠ database access. A user should be able to perform the business action exposed by the Service Catalog without automatically receiving unrestricted access to the underlying tables.

- Use least privilege: grant only the access required for the user's role.
- Protect fare/master data from unauthorized modification.
- Prevent direct manipulation of generated ticket information.
- Review ACL behavior with administrator and non-administrator test accounts.
- Use server-side authorization for sensitive operations; client-side hiding alone is not a security boundary.

8. Access Control Test Matrix

Role / User Type	Catalog Booking	Direct Table Read	Create/Write/Delete Backend Data
Client / Passenger	Allowed through catalog	Restricted unless explicitly permitted	Restricted
Operator	Allowed	Only required records/data	According to operational role
Administrator	Allowed	Administrative access	Administrative access

9. QA Testing Strategy

QA testing validates that the catalog item, client scripts, UI policies, backend data and security controls work together. Testing should include both expected user journeys and invalid or unexpected inputs.

10. Functional Test Scenarios

ID	Scenario	Expected Result
TC01	Open Book A Metro Ticket	Catalog form loads successfully.
TC02	Leave Starting From empty	Required-field validation prevents incomplete submission.
TC03	Leave Going To empty	Required-field validation prevents incomplete submission.
TC04	Select valid source/destination	Reference variables accept valid station records.
TC05	Single Journey + passenger count	Correct single fare is populated.
TC06	Return Journey + passenger count	Correct return fare is populated.
TC07	Change passenger count	Fare recalculates dynamically.
TC08	Change route after fare calculation	Old fare is cleared/recalculated according to current route.
TC09	Payment = Others	Enter Payment Mode becomes visible.
TC10	Payment = UPI/Card	Enter Payment Mode remains hidden.
TC11	First Checkout	QR modal opens and first submission is blocked.
TC12	Confirm QR flow	User can continue the intended checkout flow.

11. Negative & Boundary Testing

- Attempt submission with missing source or destination.
- Attempt a source and destination combination that is not configured for fare calculation.
- Test minimum and maximum supported passenger counts (1–4 in the current configuration).
- Change journey type after a fare has already been calculated.
- Change payment mode repeatedly between Others and UPI/Card.
- Close the QR modal and verify the checkout behavior.
- Attempt access using a role that should not have backend table permissions.
- Verify that invalid or manipulated values are not trusted solely because the client form displayed them.

12. QA Evidence

Capture screenshots or test results for successful booking, calculated fare, conditional payment field, QR modal, failed validation and restricted access. Record the test case, input, expected result, actual result and status.

13. Data Integrity

Data integrity ensures that information captured through the Service Catalog remains accurate and consistent when it reaches backend records. The project uses validation, mandatory fields, auto-population and field mapping to reduce inconsistent data.

14. Validation Rules

- Source validation: Starting From must contain a valid station reference.
- Destination validation: Going To must contain a valid station reference.
- Route validation: Fare logic should apply only to approved source/destination combinations.
- Passenger validation: Passenger count must be one of the supported choices.
- Journey validation: Journey type must be a supported choice such as Single Journey or Return Journey.
- Fare validation: Calculated fare must correspond to route, journey type and passenger count.
- Payment validation: When Others is selected, Enter Payment Mode must capture the additional payment information required by the design.
- Date validation: Where a travel date is introduced, only an approved/valid travel date should be accepted.

15. Mandatory Fields

Mandatory fields are enforced to prevent incomplete submissions. Required values should be validated at both the client and server side where the business rule is important. Client-side validation improves user experience; server-side validation protects the data layer.

Field	Why It Matters
Starting From	Defines the origin of the journey.
Going To	Defines the destination and drives route/fare selection.
Type of Journey	Determines single vs return fare.
No. of Passengers	Determines total fare.
Mode of Payment	Defines payment method.
Enter Payment Mode	Required only when Others is selected, according to the UI policy.

16. Auto-Population Logic

Catalog Client Scripts and UI Policies are used to set or display values dynamically. The fare fields are populated based on the values entered by the user, reducing manual calculation and minimizing input errors.

Source + Destination + Journey Type + Passenger Count → Fare Calculation → Fare Field

17. Field Mapping & Process Automation

The implementation uses the Process Automation Engine to map catalog inputs and populate the corresponding values. Field mapping provides the bridge between what the client enters on the Service Catalog form and what is stored or processed by the backend.

18. Mapping Approach

Catalog Variable	Mapped Business Data	Purpose
Starting From	Source station reference	Identifies journey origin.
Going To	Destination station reference	Identifies journey destination.
Type of Journey	Journey type	Controls fare calculation.
No. of Passengers	Passenger count	Controls total fare.
Amount for Single Journey	Calculated single fare	Stores/displays calculated amount.
Amount Including Return	Calculated return fare	Stores/displays return amount.
Mode of Payment	Payment mode	Captures selected payment method.
Enter Payment Mode	Additional payment mode	Captures value when Others is selected.

19. End-to-End Data Flow

Passenger Input → Catalog Variables → Client/UI Logic → Validation → Process Automation / Field Mapping → Backend Record → Ticket / QR Experience

20. User Interface Evidence — Booking Form

Figure 1. Book A Metro Ticket catalog form showing source, destination, journey type and passenger input.

The form provides the primary controlled entry point for clients. Required fields and supported choices guide the user before the request reaches backend processing.

21. Data Integrity Checklist

- No empty mandatory values
- Valid station references
- Supported passenger count
- Correct fare calculation
- Correct conditional payment field
- Mapped values reach intended backend fields
- No unauthorized direct data modification
- Historical records remain consistent

22. Test Evidence — Calculated Fare

Figure 2. Catalog form showing dynamically populated fare fields.

The screenshot demonstrates the fare fields populated in the catalog interface. During QA, the value should be checked against the configured route, journey type and number of passengers.

23. Test Evidence — QR Ticket

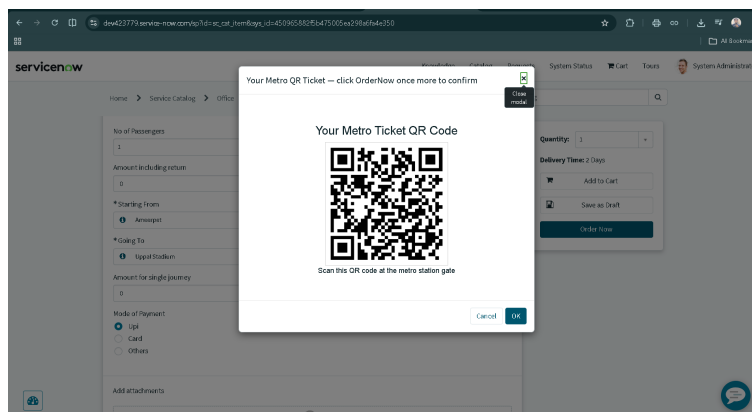


Figure 3. QR ticket modal displayed during the checkout flow.

The QR modal provides the final user-facing ticket experience. QA should confirm that it opens at the intended submission point, displays a valid QR image and does not bypass required validation or authorization.

24. Phase 4 Exit Criteria

- Access controls are reviewed and tested for intended roles.
- Catalog users can complete approved business actions without unnecessary database access.
- Mandatory and validation rules prevent incomplete or invalid submissions.
- Fare calculation is consistent across supported routes and journey types.
- UI policies and client scripts behave correctly after value changes.
- Field mapping and Process Automation populate the intended backend values.
- QR-ticket generation works through the intended checkout flow.
- Positive, negative and boundary QA scenarios are documented and passed.
- Data integrity checks confirm consistent master and transactional data.
- The solution is ready for Phase 5: Deployment, Documentation & Final Presentation.

25. Final Outcome

Phase 4 establishes the Metro Ticket Generation System as a validated, controlled and secure ServiceNow solution. The combination of ACL review, QA testing, validation, auto-population and field mapping ensures that the user experience is reliable while backend data

remains protected and consistent.